

Chapitre n° 6

TRIS

Un algorithme de tri est un algorithme qui permet d'organiser une collection d'objets selon un ordre déterminé. Le tri permet notamment de faciliter les recherches ultérieures d'un élément dans une liste.

On s'intéresse ici à des méthodes de tri d'une liste de valeurs numériques. Celle-ci est implémentée sous la forme d'un tableau à une dimension. La plupart des algorithmes de tri sont fondés sur des comparaisons successives entre les données pour déterminer la permutation correspondant à l'ordre croissant des données. Nous appellerons tri comparatif un tel tri.

La complexité de l'algorithme a alors le même ordre de grandeur que le nombre de comparaisons entre les données faites par l'algorithme.

I TRIS QUADRATIQUES

1 TRI PAR SÉLECTION

Le but est simple :

- parcourir la liste en plaçant le plus petit élément à la première position
- puis on recherche le second plus petit élément que l'on va placer en deuxième position
- etc., jusqu'à ce que le tableau soit entièrement trié.

Il faut donc une première boucle pour parcourir les positions de la liste, et à l'intérieur, une algorithme de recherche du plus petit élément dans le **reste** de la liste.

L'invariant maintenu est simple : au bout de k passages en boucles $L[0:k+1]$ est trié.

On procède avec deux fonctions.

```
1 def min_partiel(L,i):
2     # détermine le minimum de L à partir de la position i
3     pos = i
4     for k in range(i+1,len(L)):
5         if L[k]<L[pos]:
6             pos = k
7     return(pos)
8
9 def tri_selection(L):
10     n = len(L)
11     for i in range(n):
12         m = min_partiel(L,i)
13         L[i],L[m]=L[m],L[i] # échange des valeurs
```

EXERCICE 1. Tester avec la liste $[3,5,1,2]$.

Terminaison : Les appels reviennent à deux boucles for imbriquées, l'algorithme se termine donc.

Correction : Certes la fonction se termine, mais le tableau final est-il trié? Pour démontrer cela nous allons utiliser un *invariant de boucle* : " Pour chaque i la liste est une permutation de la liste initiale, la liste est triée et tous les éléments de la liste $L[i+1, n]$ sont supérieurs à tous les éléments de la liste $L[0, i+1]$ ".

Pour chaque valeur de i , au plus une permutation de deux éléments distincts a lieu et elle a lieu seulement si $L[i]$ n'est pas le minimum de $L[i : n]$. C'est pourquoi après chaque passage dans la boucle externe, la nouvelle liste est une permutation de la liste initiale. Après le premier passage dans la boucle, pour i égal à 0, la liste $[0 : 1]$ ne contient qu'un élément, le minimum de la liste, qui est inférieur à tous les éléments de la liste. La propriété est donc vraie pour i égal à 0. Si après un passage pour i égal à un k quelconque, la liste est triée et tous les éléments de $L[k+1 : n]$ sont supérieurs à tous les éléments de $L[0 : k+1]$, alors au passage suivant le minimum de la liste $L[k+1 : n]$ est placé en position d'indice $k+1$. Ce minimum est supérieur à tous les éléments de la liste $L[0 : k+1]$ et inférieur à tous les éléments de la liste $L[k+2 : n]$. La propriété est donc vraie pour i égal à $k+1$.

La propriété est donc encore vraie après le dernier passage, pour i égal à $n-2$. Donc la liste $L[0 : n-1]$ est triée et l'élément d'indice $n-1$, le dernier de la liste, est supérieur à tous les éléments de la liste $L[0 : n-1]$. Donc la liste $[0 : n]$, soit toute la liste, est triée.

Coût de l'algorithme Quels que soient les éléments d'une liste de longueur n , pour chaque valeur de i , j prend les valeurs de $i + 1$ à $n - 1$, soit $n - i - 1$ valeurs. Et pour chaque valeur de j , une unique comparaison est effectuée. Donc, pour chaque valeur de i , nous avons exactement $n - i - 1$ comparaisons.

Au total, nous obtenons : $(n - 1) + (n - 2) + \dots + 2 + 1$ comparaisons, soit $n(n - 1)/2$ comparaisons. Le coût est donc de l'ordre de n^2 quelle que soit la liste de longueur n , même si elle est déjà triée. Cela signifie que le tri par sélection n'est pas très efficace. Il est cependant simple à programmer et utile dans le cas de listes ne comptant pas plus de 10^4 éléments.

À retenir : dans tous les cas, l'algorithme de tri par sélection sur une liste de n éléments a un coût quadratique en fonction de n . Le nombre de comparaisons est de l'ordre de n^2 .

2 TRI PAR INSERTION

Voici le principe de l'algorithme :

- on commence par affecter en tant que *clef* le deuxième élément de la liste ;
- on la compare à la première valeur et on l'insère avant ou après la première selon le cas ;
- à l'étape i , le i ème élément est inséré à sa place dans la partie de liste déjà triée. Il faut partir de la fin de cette partie de liste triée, c'est à dire l'élément $i - 1$, comparer les valeurs précédentes à cette i ème valeur et s'arrêter si l'on rencontre une valeur plus petite que la i ème ;
- chaque élément dont la valeur est plus grande que la valeur i est affectée à la position suivante, ainsi toutes la valeurs plus grande que la i ème « remontent »
- à la fin, on insère la i ème valeur à sa place : la liste contient alors i valeurs triées.
- On procède ainsi de suite jusqu'à la dernière valeur.

```

1 def tri_insertion(L):
2     n = len(L)
3     for i in range(n-1):
4         k = i+1 # indice de la cle
5         cle = L[k]
6         while k>0 and cle < L[k-1] :
7             L[k] = L[k-1]
8             k = k-1
9         L[k] = cle

```

EXERCICE 2. Tester à la main avec $L = [4, 4, 3, 2, 6, 5]$

Terminaison

La boucle externe est une boucle for donc le nombre de passages est déterminé et fini. La boucle interne est une boucle while. Les valeurs prises par le variant k constituent une suite d'entiers strictement décroissante incluse dans la suite des entiers de $i + 1$ à 0 . Il y a donc, pour chaque i , au plus $i + 1$ passages dans la boucle while.

Correction

Pour prouver la correction nous utilisons un invariant de boucle : "pour chaque i , la liste est une permutation de la liste initiale et la liste $L[0 : i+2]$ est triée".

Le principe de l'insertion assure que pour chaque i , la liste modifiée est une permutation de la liste initiale. Après le premier passage dans la boucle, pour i égal à 0 , l'élément $L[0]$ et la première clé, d'indice 1 , sont rangés dans l'ordre. Donc la liste $L[0 : 2]$ est triée. La propriété est donc vraie pour i égal à 0 .

Si après un passage pour i égal à un k quelconque, la liste $L[0 : k+2]$ est triée, alors au passage suivant l'élément liste $[k+2]$ est inséré à la bonne place parmi les éléments de la liste $L[0 : k+2]$ ou reste à sa place. Donc la liste $L[0 : k+3]$ est triée. La propriété est donc vraie pour i égal à $k + 1$.

La propriété est donc encore vraie après le dernier passage, pour i égal à $n - 2$. À ce moment la liste liste $[0 : n]$, c'est-à-dire la liste liste, est triée.

Complexité :

Nous avons deux boucles imbriquées. Pour une liste de longueur n , le nombre de comparaisons peut être différent suivant la liste.

Si la liste est déjà triée, pour chaque valeur de i , k prend la valeur de $i + 1$ et il y a une seule comparaison, le test $cle < L[k-1]$. La variable i prenant $n - 1$ valeurs, cela nous fait un total de $n - 1$ comparaisons. Le coût de l'algorithme est donc de l'ordre de n .

Si par contre les éléments de la liste sont rangés dans l'ordre décroissant, alors pour chaque valeur de i , k prend les valeurs de $i + 1$ à 1 soit $i + 1$ valeurs et donc $i + 1$ comparaisons.

Au total, nous avons donc : $1 + 2 + \dots + (n - 2) + (n - 1)$ comparaisons. Ce qui donne $n(n - 1)/2$ comparaisons. Le coût est donc de l'ordre de n^2 comparaisons.

On peut montrer qu'en moyenne, le coût est de l'ordre de n^2 comparaisons, comme pour le tri par sélection. Mais le tri par insertion est très intéressant si la liste est "presque triée".

À retenir : dans le pire des cas, et en moyenne, l'algorithme de tri par insertion sur une liste de n éléments a un coût quadratique en fonction de n . Le nombre de comparaisons est de l'ordre de n^2 . Mais dans le meilleur des cas, une liste déjà triée, le coût est linéaire. L'algorithme est en général efficace sur une liste " presque triée ".

II DIVISER POUR RÉGNER

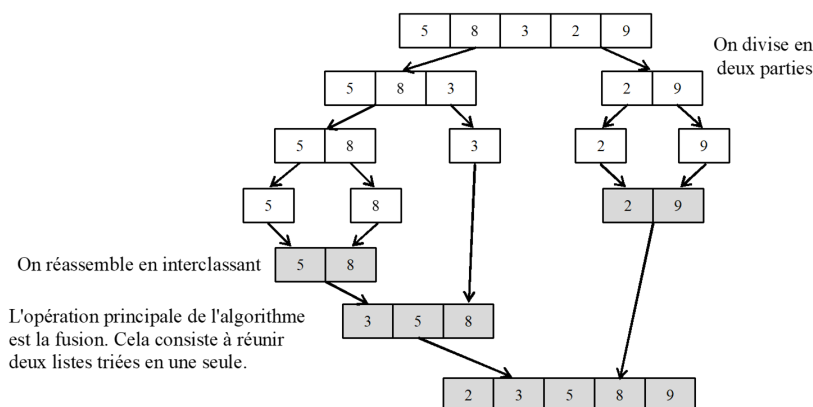
1 TRI PAR PARTITION-FUSION

La méthode de tri fusion pour un tableau de données est la suivante :

- On coupe en deux parties à peu près égales les données à trier.
- On trie les données de chaque partie par la méthode de tri fusion.
- On fusionne les deux parties en interclassant les données.

L'algorithme est donc récursif. La récursivité s'arrête car on finit par arriver à des listes composées d'un seul élément et le tri est alors trivial.

Un exemple de la méthode est donné sur la figure ci-dessous avec une liste simple :



Voici la fonction `fusion(T1,T2)` qui permet de fusionner deux tableaux triés.

```
1 def fusion(T1,T2):
2     fusion=[]
3     i,j=0,0
4     while (i<len(T1)) and (j<len(T2)):
5         if T1[i]<T2[j]:
6             fusion.append(T1[i])
7             i+=1
8         else:
9             fusion.append(T2[j])
10            j+=1
11     if len(T1)==i:
12         fusion+=T2[j:]
13     else:
14         fusion+=T1[i:]
15     return fusion
```

```
1 def Tri_fusion(T):
2     n=len(T)
3     if n==1:
4         return T
5     if n>1:
6         T1=T[0:n//2] #premiere moitie
7         T2=T[n//2:n] #deuxieme moitie
8         T1=Tri_fusion(T1)
9         T2=Tri_fusion(T2)
10        return fusion(T1,T2)
11    return T
```

Preuve de la complexité :

Si l'on s'intéresse au nombre de données à traiter à chaque appel de fonction, la relation de récurrence est du type : $C(n) = 2C(n/2) + n$

Dans le meilleur des cas, on peut poser $n = 2^p$ ce qui permet d'écrire une relation sur la suite $U_p = \frac{C(2^p)}{2^p}$:

$$\frac{C(2^p)}{2^p} = \frac{2C(2^{p-1})}{2^{p-1}} + 1 \text{ Soit } U_p = U_{p-1} + 1 \text{ ce qui caractérise une suite arithmétique de raison 1.}$$

$$\text{Ainsi : } U_p = p + 1 \text{ avec } p = \frac{\ln(n)}{\ln(2)}$$

$$\text{Ainsi : } \frac{C(2^p)}{2^p} = p + 1 \text{ soit } C(n) = n \cdot \frac{\ln(n)}{\ln(2)} + n.$$

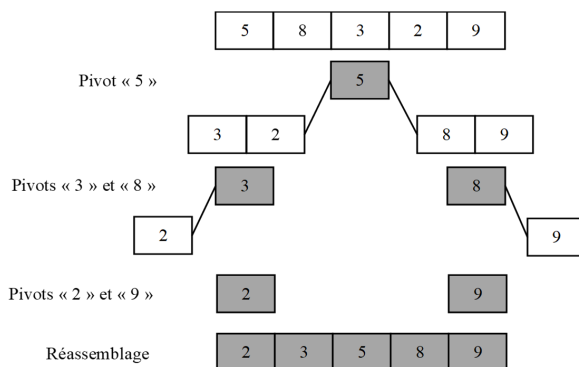
Donc la complexité est quasi linéaire $C(n) = O(n \cdot \ln(n))$

2 TRI RAPIDE

A chaque appel de la fonction de tri, le nombre de données à traiter est diminué de un. C'est-à-dire que l'on ne traite plus l'élément appelé « pivot » dans les appels de fonction ultérieurs, il est placé à sa place définitive dans le tableau. Le tableau de valeurs est ensuite segmenté en deux parties :

- dans un premier tableau, toutes les valeurs numériques sont inférieures au « pivot »,
- dans un second tableau, toutes valeurs numériques sont supérieures au « pivot ». L'appel de la fonction de tri est récursif sur les tableaux segmentés.

Un exemple de la méthode est donné sur la figure ci-dessous avec une liste simple :



```

1 def tri_rapide(L) :
2     if L == [] :
3         return [] #Le tri rapide d'une liste vide renvoi une liste vide
4     pivot = L[0]
5     listebasse=[]
6     listehaute=[]
7     for i in range (1,len (L)) :
8         if L[i] < pivot :
9             listebasse.append(L[i])
10        else :
11            listehaute.append(L[i])
12    return (tri_rapide(listebasse) + [pivot] + tri_rapide(listehaute))

```

EXERCICE 3. Tester ce tri.

Cette version du tri rapide prend le premier élément comme pivot. Ce qui peut ne pas être optimal dans certains cas. On fera la modification en TP.

Preuve de la complexité :

Le coût temporel de l'algorithme de tri est principalement donné par des opérations de comparaison sur les éléments à trier. On raisonne donc sur le nombre de données à traiter pour l'analyse de la complexité de l'algorithme.

- Dans le pire des cas, un des deux segments est vide à chaque appel de la fonction de tri. Cela arrive lorsque le tableau est déjà trié. Le nombre de données à traiter pour le i ème appel, est $n-i+1$.

On peut écrire une relation de récurrence du type $C(n) = C(n-1) + n - 1$. Le nombre total pour n appels de fonction est donc $\frac{n(n-1)}{2}$ (somme des termes d'une suite arithmétique. La complexité est donc de classe quadratique $C(n) = O(n^2)$.

- Dans le meilleur des cas, les deux segments sont de taille égale. Pour un nombre de données à traiter n , chacun des segments suivant a donc au plus $\frac{n-1}{2}$ éléments (on retire le pivot). On répète ainsi la segmentation des tableaux jusqu'à arriver au plus à un seul élément. On peut écrire une relation de récurrence du type $C(n) = 2C(\frac{n-1}{2}) + n - 1$. Comme vu précédemment dans le tri fusion, la complexité est donc de classe quasi linéaire $C(n) = O(n \cdot \ln(n))$.